

Middleware y Manejo de Errores en FastAPI

Objetivo de la guía

Comprender el uso de middlewares en FastAPI, implementar manejo global de errores, respuestas personalizadas y excepciones propias para mejorar el control y comportamiento de una API.

Actividad de Reflexión Inicial

Cuando una aplicación recibe solicitudes de usuarios, pueden ocurrir diferentes situaciones: datos incorrectos, accesos no permitidos, rutas inexistentes y fallos inesperados. Además, algunas aplicaciones necesitan ejecutar procesos automáticos en cada petición, como registrar actividad (logging) o permitir conexiones externas (CORS).

Responda:

1. Si una app se cierra de golpe sin avisar qué falló, ¿qué pensaría de ella y ud qué haría?
RTA: Pensaría que la aplicación no es confiable porque no informa claramente los errores. Intentaría revisar los registros del sistema o contactar al soporte técnico para identificar la causa del problema.
2. Si una app le muestra un código raro (ej: Error 0x85) en vez de una instrucción clara, ¿cómo le afectaría?
RTA: Me dificultaría entender qué ocurrió y cómo solucionarlo. Los mensajes claros ayudan a los usuarios a corregir errores rápidamente.
3. Si fuera el dueño de un sistema, ¿por qué es vital registrar qué usuario hizo cada acción y a qué hora?
RTA: Porque permite realizar auditorías, mejorar la seguridad, identificar errores y conocer quién realizó cada operación dentro del sistema
4. ¿Qué le da más seguridad: leer "Contraseña incorrecta" o un "Error 401"? ¿Por qué?
RTA: "Contraseña incorrecta" porque proporciona información clara y fácil de entender para el usuario.
5. ¿Por qué cree que un servidor bloquea por defecto las peticiones que vienen de páginas web desconocidas?
RTA: Para evitar accesos no autorizados, proteger la información y reducir riesgos de ataques informáticos.

Actividad de Contextualización

FastAPI permite interceptar solicitudes usando middlewares y controlar errores mediante excepciones personalizadas.

Ingresa al siguiente recurso: [Reto middleware](#) y basado en lo que seleccione, responda las preguntas:

1. ¿Qué es un middleware?
RTA:
2. ¿Para qué sirve CORS?
RTA:
3. ¿Qué sucede cuando ocurre un error en una API?
RTA:
4. ¿Qué ventajas tiene personalizar respuestas de error?
RTA:
5. ¿Qué significa el código de estado HTTP 404 devuelto por una API?
RTA:

Apropiación del Conocimiento

¿Qué es un Middleware?

Un middleware es un proceso que se ejecuta antes o después de una solicitud HTTP. Sirve para: registrar peticiones, controlar acceso, agregar seguridad y configurar comunicación entre sistemas

Flujo básico

Solicitud → Middleware → Endpoint → Respuesta

Middleware en FastAPI

Se implementa mediante decoradores o configuraciones integradas.

Middleware de logging

Permite registrar solicitudes.

```

from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    print(f"Ruta visitada: {request.url}")

    response = await call_next(request)

    return response

```

Explicación

- request.url obtiene la ruta
- call_next() continúa la solicitud
- se imprime información en consola

¿Qué es CORS?

CORS significa: Cross-Origin Resource Sharing. Permite que aplicaciones externas puedan consumir la API.

Ejemplo:

Una aplicación React consume una API FastAPI.

Configurar CORS

```

from fastapi.middleware.cors import CORSMiddleware
from fastapi import FastAPI

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)

```

Explicación

Configuración	Función
allow_origins	dominios permitidos

allow_methods	métodos HTTP permitidos
allow_headers	cabeceras permitidas
allow_credentials	autenticación

¿Qué es el manejo de errores?

Permite controlar errores y devolver mensajes claros.

HTTPException

FastAPI usa HTTPException.

```
from fastapi import HTTPException

raise HTTPException(
    status_code=404,
    detail="Usuario no encontrado"
)
```

Ejemplo práctico

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

usuarios = [1, 2, 3]

@app.get("/usuarios/{id}")
def buscar_usuario(id: int):

    if id not in usuarios:
        raise HTTPException(
            status_code=404,
            detail="Usuario no encontrado"
        )

    return {"usuario": id}
```

Respuesta generada

```
{  
  "detail": "Usuario no encontrado"  
}
```

Manejo global de errores

Permite controlar errores personalizados en toda la aplicación.

Crear excepción personalizada

```
class UsuarioNoExiste(Exception):  
    pass
```

Crear manejador global

```
from fastapi.responses import JSONResponse  
  
@app.exception_handler(UsuarioNoExiste)  
async def manejar_error(request, exc):  
    return JSONResponse(  
        status_code=404,  
        content={  
            "mensaje": "El usuario no existe"  
        }  
    )
```

Uso práctico

```
@app.get("/cliente/{id}")  
def cliente(id: int):  
  
    if id != 1:  
        raise UsuarioNoExiste()  
  
    return {"cliente": "Freddy"}
```

Respuesta personalizada

```
{  
  "mensaje": "El usuario no existe"  
}
```

Buenas prácticas

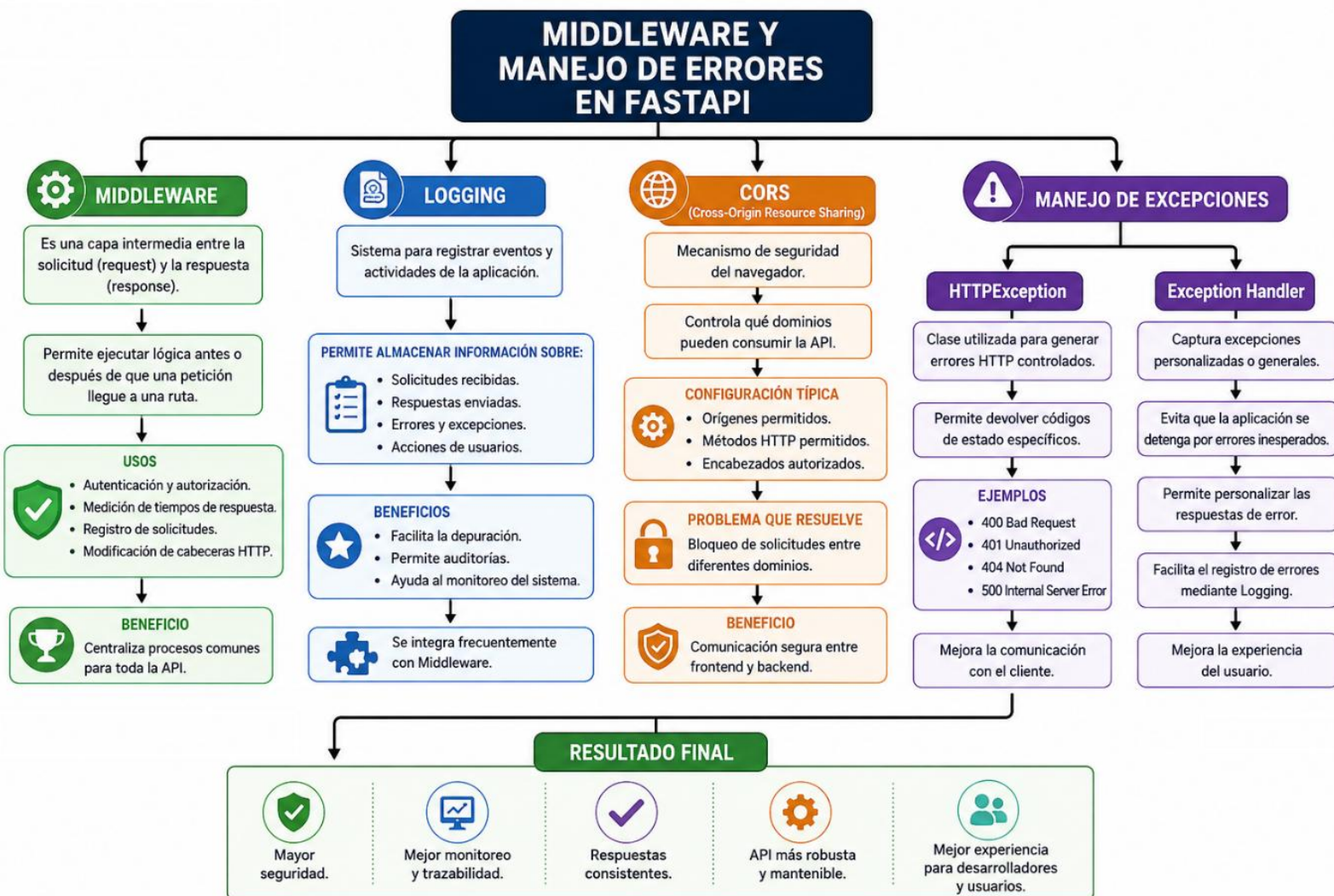
- Personalizar errores importantes
- Usar códigos HTTP adecuados
- Evitar mensajes técnicos innecesarios
- Usar middlewares simples
- Probar errores frecuentes

Actividad de Apropiación del Conocimiento

Actividad 1 – Mapa conceptual

Genere un mapa conceptual que incluya los siguientes términos(si no tiene clara la estructura consulte esta [imagen](#)):

- Middleware
- Logging
- CORS
- HTTPException
- Exception Handler



Actividad 2 – Middleware práctico

Cree un middleware que:

- imprima la URL visitada
- imprima el método HTTP (GET, POST, etc.)
- Pruebe varias rutas.

Archivo [main.py](#)

```
from fastapi import FastAPI, Request

# Inicializamos la aplicación
app = FastAPI()

# Ruta GET Usuarios
@app.get("/usuarios")
async def usuarios(request: Request):
    return {
        "url_visitada": str(request.url),
        "metodo_http": request.method
    }

# Ruta POST Equipos
@app.post("/equipos")
async def equipos(request: Request):
    return {
        "url_visitada": str(request.url),
        "metodo_http": request.method
    }

# Ruta GET con parámetro dinámico
@app.get("/proyectos/{proyecto_id}")
async def proyectos(proyecto_id: int, request: Request):
    return {
        "url_visitada": str(request.url),
        "metodo_http": request.method,
        "detalle": f"Proyecto con ID {proyecto_id}"
    }
```

Actividad 3 – Configuración CORS

Cree una API con FastAPI que cumpla con los siguientes requisitos:

- Configure CORS (Cross-Origin Resource Sharing) para permitir que aplicaciones externas (por ejemplo, una SPA en React) puedan consumir la API sin problemas.

Explique:

- Qué función cumple CORS en una API.
- Qué parámetros se configuraron en la implementación.
- Pruebe la API con al menos dos rutas.

Archivo: [main.py](#)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="API Restringida con CORS")

origins = [
    "https://mi-estudiante-react.com"
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["POST"], # Solo POST permitido desde navegador
    allow_headers=["*"],
)

@app.post("/api/datos")
def guardar_datos():
    return {
        "status": "éxito",
        "mensaje": "Petición POST aceptada por CORS"
    }

@app.get("/api/usuarios")
def listar_usuarios():
    return {
        "status": "error_cors",
        "mensaje": "El navegador puede bloquear esta respuesta por política CORS"
    }
```


Transferencia del Conocimiento

Actividad 1 – Middleware Empresarial

Cree una API en FastAPI que cumpla con los siguientes tres puntos:

1. Middleware: Intercepte cada petición e imprima en consola: la ruta visitada, el método HTTP y la fecha/hora.
2. Manejador de Errores: Cree un `exception_handler` global que capture los fallos (`HTTPException`) y devuelva un JSON personalizado con: `error: true`, el detalle del mensaje y la hora del fallo.
3. Pruebas: Diseñe dos rutas: `/api/bien` (que responda con éxito) y `/api/mal` (que lance un error 400 para probar el manejador).

Archivo [main.py](#)

```
import datetime
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import JSONResponse
app = FastAPI(title="Middleware y Errores Simples")
# =====
# 1. MANEJADOR DE ERRORES GLOBAL
# =====
# Captura cualquier HTTPException del código y lo responde de forma personalizada
@app.exception_handler(HTTPException)
async def mi manejador de errores(request: Request, exc: HTTPException):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "error": True,
            "mensaje_personalizado": exc.detail,
            "hora_error": datetime.datetime.now().strftime("%H:%M:%S")
        }
    )
```

```
# =====
# 2. MIDDLEWARE (LOGGING REQUERIDO)
# =====
@app.middleware("http")
async def mi_primer_middleware(request: Request, call_next):
    # 1. Capturar los 3 datos solicitados
    ruta = request.url.path
    metodo = request.method
    fecha_hora = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # 2. Mostrarlos en la consola/terminal
    print(f"-> [PETICIÓN] Hora: {fecha_hora} | Método: {metodo} | Ruta: {ruta}")

    # 3. Dejar que la petición continúe su camino
    respuesta = await call_next(request)
    return respuesta

# =====
# 3. RUTAS DE PRUEBA
# =====
@app.get("/api/bien")
def ruta_exitosa():
    return {"mensaje": "Todo salió perfecto"}
@app.get("/api/mal")
def ruta_con_error():
    # Lanzamos un error común, el manejador de arriba se encargará de darle el formato nuevo
    raise HTTPException(status_code=400, detail="Faltan datos importantes en la petición")
```

Actividad 2 – API Segura y Controlada

Desarrolle una API que tenga:

- CORS configurado
- mínimo 3 errores personalizados
- respuestas JSON amigables
- Realice pruebas de error.

Actividad 3 – Proyecto Formativo

Implemente en su proyecto productivo, por cada una de las rutas que ha generado:

- Mínimo 1 middleware
- CORS configurado
- Manejo global de errores
- Mínimo 3 excepciones personalizadas

Ahora realice lo siguiente en su repositorio de Github:

Crear el Pull Request (PR): Antes de fusionar, debe indicarle a GitHub qué rama quiere unir con cuál.

1. Ir a la pestaña de Pull Requests: En la barra superior de tu repositorio. Entra a tu repositorio en GitHub y haz clic en la pestaña "Pull requests" (al lado de Code). Luego, presiona el botón verde "New pull request".
2. Seleccionar las ramas a comparar: Paso clave para evitar conflictos. Verá dos menús desplegables: base: Es la rama que recibirá los cambios (normalmente main o master). compare: Es la rama donde hizo las modificaciones (la rama secundaria). Nota: GitHub te mostrará un mensaje en verde que dice "Able to merge" si no hay conflictos de código.
3. Crear la solicitud: Dar apertura al proceso. Haz clic en el botón verde "Create pull request".
4. Escribir los detalles del PR: Título y descripción. Asigna un título descriptivo a tu solicitud (por ejemplo: Feat: Agregar formulario de contacto) y detalla brevemente qué cambios incluye. Finalmente, haz clic en "Create pull request" abajo a la derecha.

Link: https://github.com/Rubloxil/DOCUMENTATION_P_SGP_Sena.git